A Design Space Exploration of Async/Await

ANONYMOUS AUTHOR(S)

Many modern programming languages include some form of asynchronous programming. In particular, a growing number now have what we call straight-line asychrony: attempts to provide asychronous functions that look similar to synchronous functions, thereby enabling asychrony without introducing complex control. These languages often share construct names like "async" and "await," which suggests that they have deep semantic similarities. Yet, a close examination reveals that these languages are quite different along several dimensions, often subtly. These differences have real semantic consequences: similar-looking programs can exhibit divergent behavior, confusing developers and language designers alike.

This paper therefore presents a *design space exploration* of straight-line asychrony. We dissect several existing languages, and show how no two of them agree as a whole on designs decisions which affect the presence and ordering of execution. We articulate a design space with nine dimensions covering the full lifecycle of an asynchronous computation, covering questions such as: What precise guarantees does a language give upon calling an asynchronous function? What happens at the end of a task's life? How can a task handle being cancelled? We explore these questions through concrete examples, informal design discussion, and a formal semantics. Our ultimate goal is to help programmers, language designers, and language theorists all better understand the emerging landscape of straight-line asynchrony.

ACM Reference Format:
Anonymous Author(s). 2026. A Design Space Exploration of Async/Await. 1, 1 (March 2026), 31 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software developers have long struggled to write concurrent and parallel programs that are both correct and efficient. Programming languages have responded to this challenge by gradually lifting elements of concurrency from user-space to the level of language design. Approaches vary greatly, but one core idea shared between most modern languages is *async/await*: marking blocks of code as `async`, and using an `await` keyword to wait for an async block to finish executing. These features can be found in recent versions of Python, JavaScript, Swift, Rust, C♯, and more.

Our original motivation for studying async/await was to help developers translate their understanding of the concept from one language to another, e.g., from JavaScript to Python. In theory, both languages contain the same core concept with small differences in the details. We should be able to explain async/await just as one might explain lexically-scoped variables, if-statements, or other standardized language features [23]. In reality, the surface similarities between async/await designs can mask deep semantic divergences. For instance: what's the difference between a coroutine, a future, a promise, and a task? What happens when you cancel a task? When are tasks required to exit, if ever? How do errors propagate across a task boundary, if it all? These questions are further complicated by languages with customizable async/await runtimes (e.g., Rust and Python), which give different answers based on the choice of runtime.

As a concrete example, consider the pseudocode in Figure 1, showing two ways to run a background task that writes information to a log. Languages differ with respect to what happens with tasks at the end of their spawning scope. Languages also differ in terms of when and how running tasks are stopped in the event of cancellation. As shown in Figure 1c, when comparing seven modern approaches to async/await, *no* two of them exhibit the same behavior on three straightforward calling contexts!

2026. ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
async fn write_to_log():
  print("A")
  // simulate log write
  await sleep(2)
  print("B")

async fn process_await():
  task = spawn write_to_log()
  // do other work ...
  await task

async fn process_detach():
  spawn write_to_log()
  // do other work ...
```

(a) Two examples of functions that spawn a task to write data to a log, one that waits for the task, and another that doesn't.

```
async fn ex1():
  await process_detach()

async fn ex2():
  task = spawn process_await()
  await sleep(1)
  task.cancel()

async fn ex3():
  task = spawn process_detach()
  await sleep(1)
  task.cancel()
```

(b) Three examples of invocations of the processing functions. The latter two use `cancel` to try and prevent the task from completing.

	ex1	ex2	ex3
Rust			
Tokio	AB	AB	AB
Smol	A	ε	ε
Python			
Asyncio	AB	ε	AB
Trio	AB	A	AB
C#	A	-	-
JavaScript	AB	-	-
Swift	A	A	A

(c) A table of outcomes when running the programs translated from the pseudocode into different languages. ε indicates no output. '-' indicates that a language does not provide a built-in `.cancel()` method.

Fig. 1. An example of an asynchronous programming pattern that exhibits different behavior under seven modern async/await languages and runtimes.

Observing this conceptual chaos, we set out to perform a *design space exploration* of async/await in modern languages. The contribution of this paper is the results of that exploration, articulated in three parts:

(1) **We start by situating the work with a high-level overview of asynchronous programming** (Section 2). We contrast asynchrony against concurrency and parallelism, and introduce common asynchrony mechanisms such as racing and cancellation.
(2) **We describe the core design dimensions of asynchrony designs in modern languages** (Section 3). Our goal is to capture all of the most fundamental distinctions between designs, with a focus on design choices that cause similar-looking programs to exhibit divergent semantics (not just performance).
(3) **We provide a precise description of each design dimension using operational semantics** (Section 4). We give a precise model of each language's async design in terms of variants on a core asynchrony calculus with delimited continuations.

2 Core Concepts in Asynchrony

Async/await are just keywords — what is the underlying concept being expressed across these languages? The obvious answer is *asynchronous programming* or, more simply, *asynchrony*. Like the words concurrency and parallelism, asynchrony does not have a definitive meaning. Rather, its meaning is socially constructed out of the types of programs people want to write. Fifteen years ago, asynchronous programming was "characterized by many simultaneously pending reactions to internal or external events" [42]. Today, we might add to that list: running I/O-bound computations in parallel, or having fine-grained control over the execution of computations. The scope of asynchronous programming has expanded.

Moreover, async/await is only one form of asynchrony. The job of async/await is to avoid the inversion of control associated with callbacks and event loops. The goal, as the C# documentation

says, is "to enable code that reads like a sequence of statements, but executes in a more complicated order" [45]. Async/await is about leaning on the language to simplify control-flow patterns, as opposed to implementing asynchrony abstractions entirely in userspace.

We concretize these points to provide some context on how asynchrony generally, and async/await specifically, fit into the broader picture of language design and implementation.

2.1 Asynchrony vs. Concurrency vs. Parallelism

We will define concurrency as a logical property of a computation, whereby two subcomputations can execute interleaved: one computation begins before another ends. Parallelism is a physical property of a computation, whereby two subcomputations can execute simultaneously: both compuations overlap at the same point in time. Parallelism implies concurrency but not vice versa. For instance, an embedded operating system running on a single core is concurrent but not parallel.

We think of asynchrony as a special case of cooperative concurrency. In an asynchronous program, two subcomputations execute interleaved because they are written to yield control to one another. Asynchrony contrasts with competitive concurrency, where interleaving occurs through external imposition, e.g., by context switches. Competitive concurrency usually arises at an architectural level below the language, such as operating system threads.

Concurrency and parallelism have programming paradigms. Paradigms of concurrent computing include shared-memory concurrency (e.g., POSIX C), communicating sequential processes (Open MPI, Go), and actors (Erlang). Paradigms of parallel computing include fork-join parallelism (Cilk, OpenMP), collection parallelism (MapReduce, Spark), and index-space parallelism (OpenGL, CUDA). Like all paradigms, these labels are useful but leaky, such as how a complex parallel program often involves shared-memory concurrency. Ultimately, a paradigm is about focusing a set of related mechanisms (`spawn`, mutexes) on a set of related problems (sorting a list, running a database).

Asynchrony also has paradigms. Callback asynchrony uses higher-order functions to provide a continuation for the completion of a computation, most notably found in C♯, JavaScript, and Swift. Event-loop asynchrony structures a program around a main loop that dispatches reactions to an event stream, most notably found in C++ and Rust. Languages have historically developed async/await to address the ergonomic problems associated with these two asynchrony paradigms. More generally, a key motivation is to provide a surface syntax for concurrent code that looks comparable to non-concurrent (synchronous) code. For example, below are excerpts from async/await proposal documents across a few languages (see Section A.2 for additional examples):

> **(C♯)** Notice first how similar it is to the synchronous code. . . . The control flow is completely unaltered, and there are no callbacks in sight. That doesn't mean that there are no callbacks, but the compiler takes care of creating and signing them up . . . [44]

> **(Python)** It is proposed to make coroutines a proper standalone concept in Python, and introduce new supporting syntax. The ultimate goal is to help establish a common, easily approachable, mental model of asynchronous programming in Python and make it as close to synchronous programming as possible. [35]

> **(Rust)** In brief, an asynchronous function returns a future, rather than evaluating immediately when it is called. Inside the function, other futures can be awaited using an await expression, which causes them to yield control while the future is being polled. From a user's perspective, they can use async/await as if it were synchronous code, and only need to annotate their functions and calls. [49]

```javascript
function timeout<T>(
  duration: number,
  f: () => Promise<T>
): Promise<T | null> {
  return Promise.race([
    f(), sleep(duration),
  ]);
}
```

(a) JavaScript.

```python
async def timeout(
  duration: float,
  f: Callable[[],Awaitable
     [T]]
) -> T | None:
  with trio.move_on_after(
    duration
  ):
    return await f()
  return None
```

(b) Python+Trio.

```rust
async fn timeout<T, Fut>(
  duration: Duration,
  f: impl FnOnce() -> Fut
) -> Option<T>
where Fut: Future<Output = T> {
  tokio::select! {
    result = f() => Some(result),
    _ = sleep(duration) => None,
  }
}
```

(c) Rust+Tokio.

Fig. 2. A simplified implementation of `timeout`, demonstrating primitives for racing.

2.2 Straight-line Asynchrony

We therefore define the paradigm embodied by async/await as *straight-line asynchrony*, given its fundamental organizing principle of eliminating complex control flow. An alternative name would be *task asynchrony*, used within the C♯ community. As we will discuss, tasks are indeed a central concept to async/await, but when looking cross-linguistically, tasks are not the only element of straight-line asynchrony. For instance, languages like Rust provide coroutines so straight-line asynchrony can be achieved without ever creating a task. Some designs like Python+Trio do not expose any concept of a task to the user.

The goal of this paper is to explore the design space of straight-line asynchrony. We primarily focus on programming languages that (a) have a relatively settled design for straight-line asynchrony, and (b) are in relatively widespread use. That includes languages with a singular form of asynchrony: JavaScript (specifically ECMAScript 2025, with Typescript types for clarity), C♯ (v14 on .NET 10), and Swift (v6.2). It also includes languages with a modular form of asynchrony: Python (v3.14, using Asyncio and Trio v0.33) and Rust (v1.92.0, with Tokio v1.50.0 and smol v2.0.2). We discuss how our findings relate to other languages in Section 6.

We describe our findings informally in Section 3 and formally in Section 4. We do not expect our readers to be familiar with the concrete syntax of all relevant languages, so we articulate the design dimensions at a high level, and present code examples primarily using the same pseudocode style in Figure 1. However, it is useful to first get a sense for both the gestalt and the variety of syntaxes, mechanisms, and semantics in straight-line asynchrony. We therefore conclude this section by implementing an async utility in multiple languages.

2.3 A Case Study on Timing Out

Consider implementing a `timeout` function, which takes as input an asynchronous function and a duration. The minimal semantics is that `timeout` returns either the result of the asynchronous work, or if the work takes longer than the duration, an indication of timing out. The `timeout` function is a paradigmatic example of asynchronous programming, as its implementation is often non-trivial when only using primitives from other concurrency or parallelism paradigms.

Figure 2 shows three implementations of the minimal `timeout` semantics. Most straight-line async designs include a race primitive (also called `select`, `or`, `wait`, `next`) which returns the first of several asynchronous computations to complete, as shown in the JavaScript and Rust+Tokio examples. The dual of racing is to wait for all computations to complete, variously called `all`, `and`, or `join`. Notably,

```rust
async fn timeout<T, Fut>(
  duration: Duration,
  token: CancellationToken,
  f: impl FnOnce(CancellationToken) -> Fut,
) -> TimeoutResult<T>
where
  T: Send + 'static,
  Fut: Future<Output = T> + Send + 'static,
{
  let mut handle = tokio::spawn(
    f(token.child_token()));
  tokio::select! {
    result = &mut handle =>
      TimeoutResult::Success(result.unwrap()),
    _ = token.cancelled() =>
      TimeoutResult::Cancelled,
    _ = sleep(duration) => {
      token.cancel();
      TimeoutResult::TimedOut
    }
  }
}
```

(a) Rust+Tokio.

```swift
func timeout<T: Sendable>(
  duration: Duration,
  f: @Sendable () async throws -> T
) async throws -> T? {
  try await withoutActuallyEscaping(f) {
    f in
    try await withThrowingTaskGroup {
      group in
      group.addTask { try await f() }
      group.addTask {
        try await Task.sleep(
          for: duration)
        return nil
      }
      defer { group.cancelAll() }
      return try await group.next()!
    }
  }
}
```

(b) Swift.

Fig. 3. An enhanced implementation of `timeout`, which handles cancellation both from within and without the function.

some designs like Trio offer a different set of primitives, so `move_on_after` is a timeout primitive which cannot be implemented by the user.

An enhanced semantics for `timeout` should take into account *cancellation*. Cancellation is another core concept for asynchrony, which did not feature prominently in earlier designs such as JavaScript and C#, but is more consciously part of later designs such as Swift and Python. For `timeout`, we desire two properties. First, if a computation exceeds the duration, the computation is cancelled. Seconds, if the `timeout` is cancelled, then its child async computations are also cancelled. The cancellation should happen such that the child computation can execute cleanup code before terminating.

Figure 3 shows two implementations of the enhanced `timeout` semantics. (See Section A.3 for additional implementations in other languages.) The Rust+Tokio function uses an explicit data structure, the `CancellationToken`, threaded throughout the code. This token is a signal for a set of related tasks to end. To enable `timeout` to cancel its child computation, the token is passed to `f`, which should (by convention!) react to the token. To enable a caller to cancel `timeout`, the token's cancellation event is handled in the `select!` expression. This style of explicit cancellation is akin to that found in JavaScript (via `AbortController` and `AbortSignal`) and C# (via `CancellationToken`). Note that one could alternatively implement the Rust+Tokio code without a cancellation token by calling `handle.abort()`, which terminates the computation at the next await point. This strategy would still invoke Rust's standard `Drop` handlers that precede memory deallocation, but it would not permit for any more customized cleanup code.

In Swift, cancellation is more deeply integrated into its asynchrony design. Asynchronous work is encapsulated in a core `Task` data structure, which contains a signal for whether it has been

cancelled. Swift propagates the signal between associated tasks — for example, the `timeout` task awaits on `withThrowingTaskGroup`, causing an association between the tasks. Similarly, the task group is associated with its children. Cancelling `timeout` propagates inwards. By convention, asynchronous computations which observe the cancellation signal (e.g., `Task.sleep`) will throw a `CancellationError`. Hence, cancelling `timeout` causes an exception to be thrown inside both `sleep` and potentially `f`, which propagates outwards outside of `timeout`.

In sum, Figure 1 demonstrated how different designs for straight-line asynchrony can take the same simple-looking program and cause different outcomes. This section demonstrated how for more sophisticated asynchronous patterns, the same concept can require substantively different implementations. Together, they motivate the need for a better understanding of the collective design space of straight-line asynchrony.

3 Design Dimensions

Every design for straight-line asynchrony is composed of dozens of individual design decisions that trade-off performance, correctness, usability, extensibility, interoperability, and so on. To scope this paper, we focus on design decisions that substantively influence the *functionality* of asynchronous programs. As suggested by Figure 1, we want to explain how similar-looking programs can have divergent behavior in terms of the order or presence of operations.

By contrast, this paper does not attempt to account for design decisions that are principally about *ergonomics*: for example, should `await` be a prefix or postfix operator in the concrete syntax? (We would argue postfix, but we will save that argument for another time.) This paper also does not focus on design decisions that are principally about *performance*: for example, how should an asynchronous runtime efficiently interface with the operating system? However, for straight-line asynchrony, performance and functionality are difficult to disentangle. Several design dimensions we discuss will be *motivated* by performance, but the key criterion for inclusion is their ultimate impact on functionality.

To perform this design space exploration, we manually analyzed the designs of today's languages containing straight-line asynchrony features. We ultimately focused on the languages enumerated in Section 2.2 (JavaScript, C♯, Swift, Python, Rust), but we also examined Kotlin, C++20, F♯, Haskell, OCaml, Hack, Nim, Dart, and Zig. We examined design documents (e.g., RFCs, published papers), GitHub discussions, and community blogs. In addition, we drew on reports from community members of confusing or surprising behavior in async programs [3, 22, 31–34, 36, 37, 47].

We identified eight design dimensions, collected into Figure 4. These dimensions are grouped into three categories, Start of Life, End of Life, and Cancellation, which serve to organize the remainder of this section. For each design dimension, we explain the points along that dimension, and the design justifications offered by different languages for selecting each point.

3.1 Start of Life

An asynchronous computation is generally defined through a function explicitly marked as asynchronous, such as the `async fn write_to_log` in Figure 1a. To begin invoking an asynchronous function in any language, one simply calls the function, as in `write_to_log()`. Immediately, the semantics diverge along the first axis: Eagerness.

3.1.1 Eagerness. Consider the program in Figure 5, where an async function `work` is called twice, and later awaited. Based on each language's approach to async function calls, three outcomes can occur: *lazy*, where no work occurs until being awaited, *eager*, where work occurs immediately in the main thread of execution, and *semi-eager*, where work is immediately scheduled on other

Start of Life

Eagerness

How to evaluate an async function application.

LAZY
Evaluate to a coroutine without executing further.
Python, Rust

EAGER
Evaluate in current thread, and schedule as task on await.
C#, JavaScript

SEMI-EAGER
Evaluate to task, and schedule task for execution.
Swift

Suspension

Guarantees on whether await points suspend.

STATIC
Await points guaranteed to suspend.
JavaScript

DYNAMIC
No guarantees on awaiting tasks.
Python, Rust, C#, Swift

End of Life

Extent

The default interval of time during which a task may exist.

INDEFINITE
Tasks by default may exist until the end of the runtime.
JavaScript, C#, Tokio, Smol, Asyncio

DYNAMIC
Tasks by default may exist until the end of their spawning scope.
Swift, Trio

Reference Strength

[For indefinite extent] The type of reference to a task held by the runtime.

STRONG
The runtime holds a strong reference.
JavaScript, C#, Tokio

WEAK
The runtime holds a weak reference.
Asyncio, Smol

Destruction

How a task is cleaned up at the end of its extent.

AWAITED
The task is awaited to completion.
JavaScript, Trio

CANCELLED
The task is cancelled, and then possibly awaited.
Swift, Tokio, Smol, Asyncio

TERMINATED
The program exits.
C#

Propagation

What happens to exceptions in unawaited tasks.

DESTRUCTION
Exceptions are reraised by the task parent.
Trio

NEVER
The exception is kept within the task.
JavaScript, C#, Tokio, Smol, Asyncio, Swift

Cancellation

Awareness

Whether a task is able to respond to being cancelled.

UNAWARE
The task cannot respond to being cancelled.
Rust

AWARE
The task can respond to being cancelled.
Asyncio, Trio, Swift

Direction

How cancellation is communicated through the task graph.

TOP-DOWN
Starting from the root task, and down through the children.
Rust

BOTTOM-UP
Starting from the leaf tasks, and up through the parents.
Asyncio, Trio

SIMULTANEOUS
To all nodes in the task graph at once.
Swift

Persistence

[For aware cancellation] How long a cancellation of a task lasts.

TRANSIENT
A task can ignore cancellation and proceed as normal.
Asyncio

PERSISTENT
A task can ignore cancellation but remains cancelled.
Trio, Swift

Fig. 4. Overview of the straight-line asynchrony design dimensions.

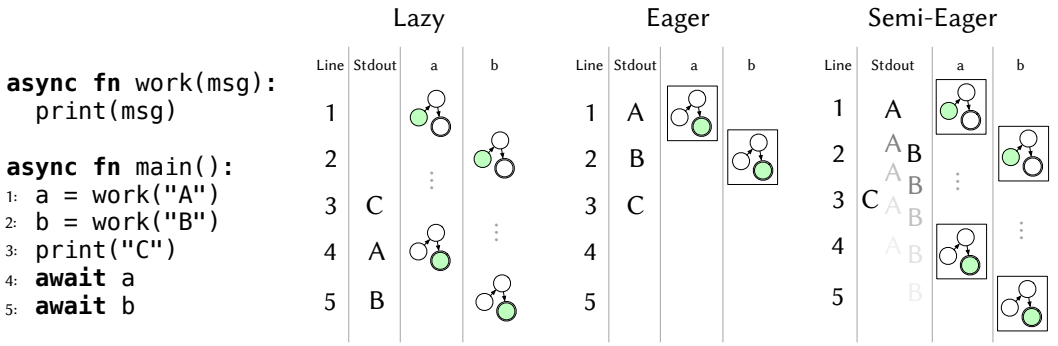


Fig. 5. An example program demonstrating the Eagerness dimension, executed with Rust (lazy), C# (eager), and Swift (semi-eager). Each column shows the line of execution, the text printed to stdout, and changes to the values of the variables a and b. Coroutines are represented as a state machine, with the active state colored in green. Tasks are represented as a boxed state machine.

threads, resulting in non-deterministic output. To understand this variation, we first need to discuss two foundational concepts for straight-line asynchrony: coroutines and tasks.

An asynchronous function either implicitly (in the runtime implementation) or explicitly (in a user-visible way) desugars into an object that behaves like a coroutine. Upon entering the function, code runs until reaching an **`await`** point, when the function may yield to its caller (depending on its Suspension, Section 3.1.2). Specifically, these objects behave like asymmetric stackless coroutines, to use the taxonomy provided by Moura and Ierusalimschy [30]. General coroutines can yield values on suspension and accept values on resumption. For straight-line asynchrony, a coroutine does not need to yield values on suspension, but needs only to accept values on resumption (or access them at a shared location).

When a coroutine representing an asynchronous computation is constructed via a function call, the Eagerness axis defines what happens next to the coroutine. In languages with first-class coroutines[1] such as Python [35] and Rust [49], we describe their behavior as *lazy*: the coroutine is returned and no work occurs. In Figure 5, this means that `a = work("A")` constructs a coroutine in a starting state, but the coroutine is only executed at **`await`** `a` on line 4. In lazy languages, to execute an async computation concurrently with its caller, one must use an explicit **`spawn`** operation to convert the coroutine into a *task* (e.g., as used in Figure 1).

A task is a handle on an async computation shared between its creator and an *async runtime*. The async runtime usually consists of a thread pool and some facilities for reacting to OS-level events by waking up waiting tasks. In languages without first-class coroutines, async function calls returns tasks directly. However, languages differ with respect to scheduling of these tasks.

For *eager* languages such as C# [38, §15.14.3] and JavaScript [10, §27.7.5.1], on evaluating an async function application, the current thread transfers execution to the body of the async function. When reaching an await point, the function application evaluates to a task value, and control returns to the caller. In Figure 5, eager execution causes `work("A")` to behave like a normal function call, resulting in deterministic/single-threaded execution.

[1]As of v1.92.0, Rust has first-class *futures* but not first-class coroutines. That is, calling an async function returns a value representing the computation, but that value does not offer a full coroutine interface. By contrast, Python has a stabilized coroutine interface implemented by its async functions. The Rust compiler does internally track async functions as coroutines, but this interface has not been stabilized.

```
async fn work(msg):
  print(msg)

async fn repeat(msg):
  await work(msg)
  await work(msg)

async fn main():
  a = spawn repeat("A")
  b = spawn repeat("B")
  print("C")
  await a
  await b
```

Static

```
a = repeat("A")
  work("A")
    print("A")          A
  await work("A")
b = repeat("B")
  work("B")
    print("B")          B
  await work("B")
print("C")              C
await a
  work("A")
    print("A")          A
  await work("A")
await b
  work("B")
    print("B")          B
  await work("B")
```

Dynamic

```
a = repeat("A")
  work("A")
    print("A")          A
  await work("A")
  work("A")
    print("A")          A
  await work("A")
b = repeat("B")
  work("B")
    print("B")          B
  await work("B")
  work("B")
    print("B")          B
  await work("B")
print("C")              C
await a
await b
```

Fig. 6. An example program demonstrating the Suspension dimension, executed with JavaScript (static) and C# (dynamic). Each column shows an execution trace of the program under an eager semantics with each kind of suspension. The execution traces are nested according to their level in the call stack, and the order of prints to stdout is emphasized in the right part of each column.

For *semi-eager* languages such as Swift [26], an async function application immediately schedules the task onto a work queue, but allows the calling thread to continue executing past the function call. In Figure 5, semi-eager execution means that `work("A")` will run at some point after being called. The task is only guaranteed to have finished by the end of **`await`** `a`, leading to nondeterministic/multi-threaded execution.

Design rationale. The choices in the eagerness dimension offer a tradeoff in predictability, maximizing CPU multi-core utilization, and reducing task allocation. Lazy asynchrony has two main benefits. First, lazy asynchrony can provide predictable memory allocation. Asynchronous function applications evaluate directly to a coroutine value that can be stack allocated. The developer must opt-in to either boxing or spawning a coroutine. Predictable memory allocation is a key motivation for Rust's async/await design [50]. Second, by separating coroutines from tasks, the language can enable users to provide their own runtime. For example in Rust, this feature is useful so the runtime can be tailored to the needs of individual domains, such as embedded systems running Embassy versus backend web servers running Tokio/Smol.

The downsides of the lazy approach include: by offering an extensible runtime, async-related libraries in the ecosystem tend to have reduced interoperability. For example, in Rust, **`spawn`** is a runtime-specific primitive, so libraries using it must be coupled to a choice of runtime such as Tokio or smol. Users must also deal with the complexity of understanding both coroutines and tasks. Users often find lazy behavior surprising — to call a function and have nothing happen runs against a programmer's intuition transferred from synchronous function calls [8, 14, 24].

Non-lazy approaches avoid these downsides by starting asynchronous work as soon as a function is called, either on the current thread (eager) or any thread (semi-eager). The eager approach reduces the overhead of control flow and context switches within the asynchrony runtime. The semi-eager approach leans more on the runtime, but in exchange provides a higher degree concurrency by dispatching work to other threads. The increased concurrency can be particularly valuable for programs that want to avoid blocking a main thread, such as in UI applications [5, 19, 48].

3.1.2 Suspension. Once asynchronous work has started, the next design divergence occurs at await points. The Suspension dimension defines the guarantees a language makes about whether an await point is guaranteed to yield control. For example, consider the program in Figure 6 executing under an eager semantics. The line `a = repeat("A")` will call into `work("A")` which will call `print("A")` and return to **`await`** `work("A")`. The task for `work("A")` has been completed, so a language has two choices: either yield control to the caller, or continue evaluation past the **`await`**. We call the former strategy *static* suspension because an await point is statically guaranteed to always yield. We call the latter strategy *dynamic* suspension because whether an await point yields depends on the awaited task.

In only one language surveyed, JavaScript, an await point is guaranteed according to the ECMAScript specification [10, §27.7.5.3, step 3.b]. Therefore in Figure 6, running under JavaScript semantics, the program outputs "ABCAB" because **`await`** `work("A")` yields back to `main`. In all other languages, no such guarantee exists. For the example program operating under an eager semantics such as in C#, the program would output "AABBC" because **`await`** `work("A")` observes that `work("A")` is complete and therefore the await expression can continue evaluation.

Design rationale. The principal reason to adopt static suspension is to prevent starvation. Under an eager approach with dynamic suspension, if a compute-bound task is called before calling I/O-bound tasks, the first task can starve the remaining tasks of CPU time. A static suspension policy reduces the likelihood of that compute-bound task never yielding control back to the executor.

Static suspension comes at a performance cost. In the case of awaiting a completed task, static suspension requires a round-trip to the runtime and back to continue past the await. Languages with dynamic suspension can mitigate the starvation issue by providing a `yield_now` primitive which has no effect but to yield control to the runtime.

3.2 End of Life

Much of the divergence between designs for straight-line asynchrony is about when and how tasks end. Recent designs often introduce some kind of "structured" asynchrony to ensure that tasks are accounted for at all times, and do not end up dangling or erroring unbeknownst the rest of the program. This subsection covers the design dimensions of Extent (how long a task lives, Section 3.2.1), Reference Strength (the type of reference to tasks held by the runtime, Section 3.2.2), Destruction (what happens to destructed tasks, Section 3.2.3), and Propagation (what happens to unawaited exceptions, Section 3.2.4).

3.2.1 Extent. Borrowing terminology from the Lisp community, the *extent* of a task is the interval of time during which that task is capable of executing. For example, consider the program in Figure 7. What is the extent of the **`spawn`** `work()` task? The answer is it depends on quite a number of factors, as previously illustrated in Figure 1.

In some designs for straight-line asynchrony, especially earlier designs, tasks by default have an *indefinite* extent: the task can live until it is no longer reachable, at which point the task is destructed. JavaScript, C#, Python+Asyncio, and Rust+Tokio/Smol all use indefinite extent. Note that designs differ on how to determine reachability (see Section 3.2.2), and designs also differ on the mechanism for destruction (see Section 3.2.3). Figure 7 illustrates indefinite extent, showing how the `work()` task outlives its parent `main()` task, and the program waits until the work is completed.

Some have called indefinite extent an "unstructured" approach to asynchrony [36], akin to using goto for control flow, as task handles can easily go unhandled, leaving asynchronous work dangling with no clear policy for managing it. This line of thought has led to the devleopment of "structured" asynchrony approaches [11, 27, 41], which we describe as having *dynamic* extent, found in Swift and Python+Trio. With dynamic extent, a task's extent is tied to the extent of another task. In Figure 7, under dynamic extent, the `work` task is associated with the `main` task. When the `main` task completes,

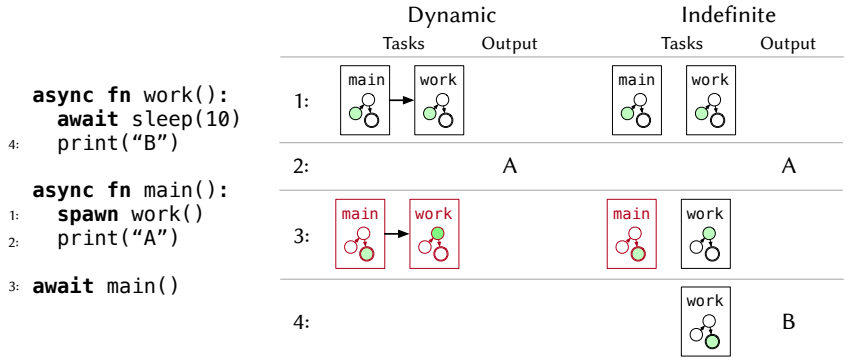


Fig. 7. An example program demonstrating the Extent dimension, executed with Swift (dynamic) and JavaScript (indefinite). Each column shows the task structure and output after executing each line. Arrows between tasks indicate a parent-child relationship used for dynamic extent.

its associated tasks are destructed, which means cancellation in the case of Swift. Therefore the second print never occurs.

Design rationale. The main argument for dynamic extent is that limiting tasks to known scopes leads to programs which are easier to reason about, and compose better with other language features [11, 27]. For instance in Python, dynamic extent can ensure that a task within a `with` block will not use resources (e.g., file descriptors) after they have been cleaned up [36]. However, existing implementations of dynamic extent only support a strictly hierarchical model of parent-child task relationships (via `TaskGroup` in Swift and `Nursery` in Trio). These implementations disallow programs which might have a more complex dependencies between tasks, such as two parent tasks awaiting the same child task.

3.2.2 Reference Strength. For designs with indefinite extent, tasks live until no longer reachable. One lever available to these designs is to ask: does the runtime's reference to a task count? In the parlance of reference counting garbage collection, is the runtime's reference *strong* or *weak*? This choice constitutes the Reference Strength dimension.

For most indefinite extent designs, the runtime's handle is strong: JavaScript, C♯, Rust+Tokio. Consequently, even if a task is no longer accessible from the main body of a program, it is still reachable by the runtime, and will therefore be executed to completion. Two designs use weak runtime handles: Rust+Smol and Python+Asyncio.

Rust lacks a garbage collector, so weak reference strength does not translate literally for Smol. In Smol, the task handle received from a **`spawn`** implements a destructor (via the `Drop` trait), called when the handle goes out of scope (unless moved). The handle's destructor destructs the task, cancelling it in the case of Smol. In this way, Smol effectively achieves dynamic extent. For example, the dynamic extent semantics for the program in Figure 7 are the same as Smol's semantics, and the indefinite semantics are the same as Tokio's semantics.

Asyncio also holds weak references to tasks. Unlike Smol's intentional design, Asyncio's weak reference approach is presently marked as a bug on the CPython repository. Interestingly, it is not currently known why Asyncio holds weak references. According to Guido van Rossum, speaking on the relevant Github issue thread [17]:

> I'm not sure why we designed it this way, and (from private email) @1st1 doesn't seem to recall either. But it was definitely designed with some purpose in mind – the

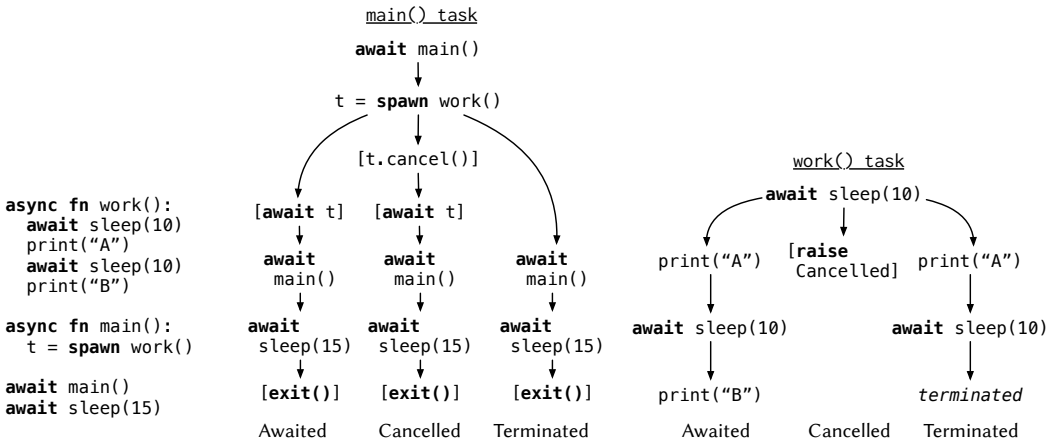


Fig. 8. An example program demonstrating the Destruction dimension. Each task diagram shows the sequence of execution of instructions, branching at the point of divergence between each Destruction approach. Administrative instructions which are executed but don't appear in the source code are displayed in square brackets.

> code is quite complex and was updated every time an issue with it was discovered, and we've had many opportunities to change this behavior but instead chose to update the docs (python/cpython/pull/29163), even when asyncio itself suffered (python/cpython/issues/90467).

3.2.3 Destruction. Once a language has decided that it is time to destroy a task, we encounter yet another point of divergence: what does it mean to destroy a task? For example, consider the program in Figure 8. At the end of the extent of the spawned `work()` task (either end of `main()` for dynamic extent, or end of program for indefinite extent), what happens to the `work()` task? Today's languages offer three possibilities: to *await* the task, to *cancel* the task, and to *terminate* the task.

The destruction dimension is orthogonal to extent. JavaScript is a language with indefinite extent and awaiting destruction — Figure 7 previously showed an example relying on this pair of design decisions. Python+Trio is a language/system with dynamic extent and awaiting destruction, as tasks are tied to a "nursery," and the nursery waits until all its child tasks are completed before exiting. The awaiting column in Figure 8 demonstrates Trio semantics, where the program would print "ABC" by waiting for the `work()` task to before exiting `main()`.

Most other designs opt to cancel and then await[2] tasks at the end of extent, such as Swift, Rust+Tokio, Rust+Smol, and Python+Asyncio. Figure 8 running under Swift would output nothing because `work()` is cancelled before it can reach a print statement. By contrast, C# will *terminate* tasks at the end of their indefinite extent, meaning the runtime simply exits while the destroyed tasks are running. In Figure 8, under C#, the program would print "A" because the tasks continues until it is terminated after 15 seconds.

[2]The one exception here is Smol, which does not await cancelled tasks under its dynamic extent by default. The reason is that Smol implements task destruction using Rust's `Drop` trait, which only permits synchronous code, so Rust async implementations cannot await the completion of a task on drop. Smol offers an async `Task::cancel` method which implements the cancel+await semantics, but this method is not called automatically. Tokio, by contrast, does implement cancel+await because it uses indefinite extent, and it is easier to have a runtime await outstanding tasks at the end of the main function. Libraries like Trio can implement await-on-destruction using Python's **`async`** `with` construct for scoped async entry/exit methods.

Design rationale. For languages with dynamic extent, the standard approach is to use cancelled destruction. The idea is that responsible async code should await on a task in the same scope in which it was spawned. If a task is still executing after its parent scope exits, this is probably unintented, and it could create surprising behavior to wait until that task is completed. However, this approach makes less sense in Trio, whose nurseries do not expose any concept of a task handle, so it is not trivial to directly await the result of a spawned task.

For languages with indefinite extent, any approach can make sense depending on the needs of the application, and it is straightforward to configure a single global runtime to adopt the desired behavior. For example, Tokio can be configured to terminate rather than await.

3.2.4 Propagation. One final point of divergence at a task's end of life is the handling of exceptions. In all surveyed languages with exceptions (Python, Swift, JavaScript, C♯), awaiting on an async function that has thrown an exception will re-raise the exception at the await point. However, what happens to an exception raised in a task that is never awaited? We call this the Propagation dimension.

Most languages *never* propagate, meaning the exception generates no behavior outside its containing task, apart from perhaps printing to the console. Python+Asyncio and JavaScript will print a stack trace for an uncaught asynchronous exception. Node.js specifically will exit with a non-zero error code, although this latter behavior is not part of the ECMAScript specification so we still categorize it as a never approach to propagation.

```python
async def fail():
  raise Exception()

async def main():
  print("A")
  async with trio.open_nursery() as n
    n.start_soon(fail)
  print("B")
```

Fig. 9. An example of destruction propagation. Trio reraises exceptions when their associated nursery scope ends.

Python+Trio is the only system which offers a different semantics, in part because tasks cannot be awaited, so the library must offer another mechanism for observing asynchronous exceptions. The example in Figure 9 (shown in Python, not pseudocode) will print "A" but not "B" because the nursery will re-raise the exception originating in the `fail` function. We describe this as a *destruction* approach to propagation. A similar program in any other system will print "A" then "B", possibly with a stack trace printed in-between.

Design rationale. The upside to the destruction approach is that an application is less likely to ignore problematic exceptions. With the never approach, if a developer does not carefully monitor their logs, a task can fail and cause confusing bugs where expected operations never occur.

The downside is that the destruction approach may run counter to either a developer's intuitions or their desired software architecture. Concurrency APIs often seek to encapsulate errors at thread or task boundaries, so errors in one thread cannot crash the whole system. For example, in a backend web server with many concurrent connections, it is likely undesirable that one exceptional connection should crash the entire server. Either way, it seems sensible that that a straight-line asynchrony API should offer *some* way of accessing information about unawaited exceptions, rather than only printing to stderr.

3.3 Cancellation

As demonstrated by the case study on `timeout` (Section 2.3), cancellation is both a key component and a complicating factor in straight-line asynchrony. A naive cancellation approach would abruptly kill a running task, but no language adopts this approach because the task may be holding a mutex, writing to a file, or otherwise in a critical region with respect to a system's invariants.

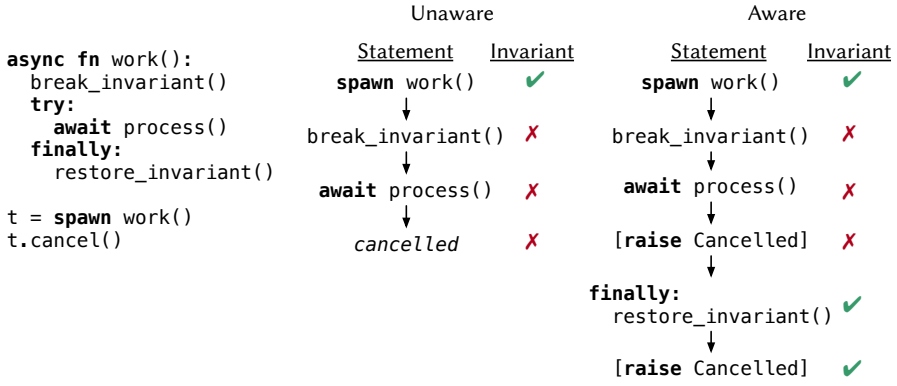


Fig. 10. Ane example program demonstrating the Awareness dimension. [TODO]

A straightforward approach to cancellation in user-space is to use a *cancellation token*, as demonstrated in Figure 3a. The developer is responsible for threading the token through all the relevant async code, and they are also responsible for reacting to the token in any async code which should be cancellable. Cancellation token APIs can be found in JavaScript, C#, and Rust. However, the amount of developer effort required for this approach is prohibitive. Modern takes on straight-line asynchrony have therefore sought to develop alternative cancellation approaches to lower the developer burden.

This subsection describes the two principal design dimensions of non-token-based, or *integrated*, cancellation approaches, Awareness (whether a task can react to cancellation, Section 3.3.1) and Direction (how cancellation propagates through related tasks, Section 3.3.2). This subsection focuses on languages with integrated cancellation designs, namely Rust, Swift, and Python.

3.3.1 Awareness. Integrated cancellation approaches all center around triggering cancellations at await points. A key distinction is whether a cancelled task is aware that it is being cancelled, in order to handle or ignore the cancellation at a given await point.

The most straightforward approach is *unaware* cancellation, where a task has no opportunity to react to being cancelled. This approach is taken by Rust in both Tokio and Smol. In Rust, cancellation essentially means never executing a coroutine again after it has yielded. With a direct handle on the coroutine, this simply means not polling the coroutine, or explicitly dropping it. If the coroutine is within a task, then the task must be marked as cancelled, and the runtime will similarly refuse to reschedule the task.

By contrast, Python and Swift both implement a form of *aware* cancellation. In Python (both Asyncio and Trio), when a task is cancelled, a cancellation exception is thrown into the task. An async function can wrap any await point in a try/except block which can catch this cancellation exception, performing cleanup or consuming the exception. In Swift, when a task is cancelled, a flag is set indicating that the task is cancelled. Functions can then opt-in to raising a cancellation exception when this flag is set.

Design rationale. A benefit of unaware cancellation is that tasks can be reliably cancelled. The principal case that delays unaware cancellation is a compute-bound task which does not cooperatively yield to the runtime. With aware cancellation, functions can either ignore cancellation or fail to opt-in, which we discuss further in Section 3.3.2.

A major drawback to unaware cancellation is that cancellation can easily violate logical invariants of a program. For example, consider the program in Figure 10. Say a program wants to do asynchronous work in a critical region where an invariant is broken and later restored. With unaware cancellation, the `work` task can be halted at **`await`** `process()`, causing the invariant to remain broken. In Rust, this is a widely-documented problem with its cancellation system[3] [32, 33]. With aware cancellation through exceptions, the `finally` block has the opportunity to restore the invariant before continuing to raise the cancellation exception.

3.3.2 Direction. Tasks form a dependency graph of parent tasks and their spawned child tasks. Cancellation generally starts at the outer task, raising the question: how does cancellation proceed through the graph? We call approaches to this question the Direction dimension.

One approach is *top-down* cancellation, where cancellation is communicated from parent to child tasks. This approach describes Rust, which leans on the `Drop` trait to implement cascading communication during cancellation. In Smol, when a task is cancelled and later dropped, that drops the handles to all of that task's children, which in turn cancels those tasks, and so on through the task graph.

Another approach is *bottom-up* cancellation, where cancellation is communicated from child to parent tasks. This approach describes Python, which leans on exceptions to communicate cancellations. When a task is cancelled, both Asyncio and Trio will walk the task graph to find its leaf nodes, and raise a cancellation exception at those tasks. That exception is then raised across await points to async callers, which can choose to either catch the exception or allow it to propagate.

The final approach is *simultaneous* cancellation, where cancellation is communicated at once to the entire task graph. This approach describes Swift, which uses a shared cancellation flag. In Swift, convention holds that async functions should observe the cancellation flag and then raise a cancellation exception. A key difference from bottom-up cancellation is that the flag ensures that a task is aware of being cancelled, even if the cancellation exception was caught and consumed.

Design rationale. The top-down approach is a straightforward means of propagating cancellation without needing a mechanism like exceptions, which is especially useful for exception-less languages such as Rust. The main limitation of the top-down approach is that it makes custom handling of cancellations — such as running cleanup code — difficult.

The bottom-up and simultaneous approaches both provide the opportunity for handling cancellations. The major difference is that the bottom-up approach provides individual async functions more leeway to ignore cancellation, irrespective of the calling context. With the simultaneous approach, every task in a task tree is guaranteed to know if it is cancelled. As implemented in Swift, this approach requires more complexity — cancellation is represented both formally through the cancellation flag, and conventionally through a cancellation exception observed by async primitives. In exchange, async functions can avoid the possibility of missed cancellations.

3.3.3 Persistence. One final element of cancellation is its Persistence: when a task is cancelled, does the cancellation request persist (e.g., subsequent async work is also marked as cancelled), or is cancellation requested once? We will describe these options as *persistent* cancellation and *transient* cancellation. The persistence dimension is important for cancel aware languages, because on cancellation a task may perform additional async work.

[3]It is possible in Rust to implement something similar to the `finally` block using the `Drop` trait. When a task is cancelled, the task is eventually dropped and drop handlers are invokes for all coroutines in the task. One can theoretically define a custom `Drop` implementation for a bespoke type that calls `restore_invariant`. However, this approach is verbose (requiring a newtype and impl), unwieldy (likely requires mutable references to data structures, complicating ownership), and synchronous-only (one cannot call async code in a `Drop` implementation without blocking.)

Swift and Python+Trio use persistent cancellation. Swift and Trio both implement a cancellation flag shared amongst a task graph, and that flag is permanently active once set. Swift's flag is part of its public API, while Trio's flag is an implementation detail. Consequently, if an async computation ignores its cancellation, then subsequent awaits may again raise a cancellation exception; for both Swift and Trio any subsequent await point may result in a cancellation exception.

Rust and Python+Asyncio use transient cancellation. In Rust's case, because cancellation is non-cooperative, cancellation is unrecoverable. For Asyncio, cancellation is recoverable, so if an async computation ignores a cancellation request, then it can continue spawning and awaiting tasks unimpeded. Importantly, tasks spawned after the initial cancellation request would be unaware of the previous cancellation attempt.

Design rationale. For non-cooperative cancellation, persistence is a moot point, so the main question is which design best fits cooperative cancellation. This perhaps comes down to a philosophy on why tasks would choose to ignore their cancellation, and what risks are incurred as a result.

The main argument for transient cancellation is for tasks to perform async resource cleanup. For example, closing a TLS socket often requires sending a `close_notify` alert to the peer so that they can be cryptographically assured that you intended to close the socket.[4] If a cancellation request persists to spawned dependencies, then the library's provided socket `.send()` method may always result in a reraised cancelled exception, with no bytes sent. Transient cancellation has a greater risk for cancellation signals to go unnoticed. If one task suppresses a cancellation signal, then all others in the graph may never become aware of the attempt.

```
with trio.move_on_after(TIMEOUT):
  conn = make_connection()
  try:
    await conn.send_hello_msg()
  finally:
    with trio.move_on_after(
      CLEANUP_TIMEOUT, shield=True
    ):
      await conn.send_goodbye_msg()
```

Fig. 11. An example of shielding async resource cleanup from persistent cancellation.

Runtimes with persistent cancellation typically come with a cancellation shielding operator to allow for async cleanup. The example in Figure 11 (shown in Python, not pseudocode), the developer uses a shielded timeout to send a closing message on the socket without being immediately recancelled. This example strikes a balance between resource managemenet and exiting promptly (via the timeout).

4 Formal Model

The preceding section provides a detailed but informal characterization of our view of the straight-line asynchrony design space. Now, we will put it on a formal footing.

We present the design dimensions as variants on a formal operational semantics for a core asynchronous language. We start with a small λ_v language that captures the computational core of Python, Rust, etc., while eliminating many language-specific details. For example, we unify the languages' implementation of asynchrony using delimited continuations rather than faithfully modeling each implementation using state machines and other such mechanisms. Atop this, we layer semantic rules that capture the essence of the design dimensions in Section 3.

Perhaps unsurprisingly, the full semantics is too large to include in its entirety. The elided rules are provided in the appendix, and a complete executable model via Redex is available in the supplementary materials.

Figures 12a and 12b provide the grammar and semantics of the core calculi that comprise our base for the async extensions. The core language λ_v contains mutable references and delimited

[4](docs.openssl.org)

$$
\begin{array}{rcl}
\text{Const } c & ::= & \text{true} \mid \text{false} \mid \text{number} \mid \text{string} \\
\text{Expr } e & ::= & x \mid v \mid \textbf{fn}\ \bar{x} \to e \mid e\ \bar{e} \\
& \mid & \text{let } x = e \text{ in } e \\
& \mid & \text{if } e \text{ then } e \text{ else } e \\
& \mid & \text{fix } e \mid e\,;\ldots;\,e \\
& \mid & \text{reset } e \mid \text{shift } x \to e \\
& \mid & \text{ref } e \mid !e \mid e := e \\
\\
\text{Value } v & ::= & c \mid () \mid \text{ptr}(x) \\
& \mid & \textbf{fn}\ \bar{x} \to e \mid \text{fix } v \\
\\
\text{Eval Ctx } E & ::= & [\cdot] \mid \bar{v}\ E\ \bar{e} \mid \text{fix } E \\
& \mid & \text{let } x = E \text{ in } e \\
& \mid & \text{let } x = v \text{ in } E \\
& \mid & \text{if } E \text{ then } e \text{ else } e \\
& \mid & \bar{v}\,;E\,;\bar{e} \mid \text{reset } E \\
& \mid & \text{ref } E \mid !E \mid E := e \\
& \mid & v := E \\
\text{Delim Ctx } M & ::= & \ldots\ (E \text{ without reset } M) \\
\text{Heap } \sigma & ::= & \cdot \mid \sigma, x \mapsto v
\end{array}
$$

(a) The grammar for λ_v.

$$
\begin{array}{rcl}
e & ::= & \ldots \mid \text{throw\_in } e \mid \text{throw } e \\
& & \mid \text{try } e \text{ catch } e_{handler} \\
\text{Exn Ctx } G & ::= & [\cdot] \mid \ldots (E \text{ without try } e \text{ catch } e)
\end{array}
$$

(b) The grammar for λ_{exn}, which extends λ_v with `try/catch`, and `throw` forms.

$$\langle \sigma, E[\text{try } G[\text{throw } v] \text{ catch } (\textbf{fn}\ x \to e)] \rangle \longrightarrow \langle \sigma, E[(\textbf{fn}\ x \to e)\ v] \rangle \quad \text{[Catch-Exn]}$$

$$\langle \sigma, G[\text{throw } v] \rangle \longrightarrow \langle \sigma, \text{throw } v \rangle \quad \text{where } G \neq [\cdot] \quad \text{[Uncaught-Exn]}$$

$$\langle \sigma, E[\text{try } v \text{ catch } e_h] \rangle \longrightarrow \langle \sigma, E[v] \rangle \quad \text{[Catch-Value]}$$

$$\langle \sigma, E[\text{throw\_in}(\textbf{fn}\ x \to E_{inner}[x], v)] \rangle \longrightarrow \langle \sigma, E[E_{inner}[\text{throw } v]] \rangle \quad \text{[Throw-In]}$$

(c) The operational semantics for the λ_{exn}.

Fig. 12. The grammar and reduction rules for the core calculi λ_v and λ_{exn}.

continuations via the `shift` and `reset` operators described by Danvy and Filinski [9]. The grammar for λ_v is provided in Figure 12a. The reduction rule is defined over a mutable heap σ and an expression e, $\langle \sigma\, e \rangle$. For space reasons, the standard reduction rules and full grammar for λ_v are available in Section A.8.

For languages with exceptions, we extend λ_v with `try/catch` and `throw` forms and refer to this language as λ_{exn}. The grammar and reduction rules for λ_{exn} are provided in Figures 12b and 12c. The λ_{exn} grammar provides the evaluation context G, which is the exception context delimited by a `try` form and discarded by the `throw` operator. We also provide a `throw_in` expression that raises an exception within a suspended computation, modeled after Python's `.throw()` coroutine method, which resumes a suspended computation with an exception instead of a value.

We now describe the configuration of the shared async grammar, which can extent any base language. Figure 13a defines the shared grammar constructs that encode the async runtime building blocks discussed in Section 3. We will refer to this extension as the async platform, λ_{async}.

A *frame*, F, is written $\langle \ell,\ e \rangle$ and consists of an expression e, along with a frame label, ℓ. A *frame label*, ℓ, is either the name of the task this frame corresponds to, or *sync* to denote a synchronous frame. A *frame stack*, FS, is a list of frames — essentially a thread. An empty frame stack is written ϵ, and we write $F \circ FS$ to denote a frame stack whose head is F and tail is FS. A *process*, P, is a collection of frame stacks. We will use the syntax $P \uplus FS$ to select one frame stack in P to run. A *task queue*, Q, stores the async work waiting to be run; Q comprises tuples that have a label and a continuation, (ℓ, κ). A *signals queue*, T, stores async work that is waiting on I/O operations, the work will be rescheduled in the future at timestampt t, at the earliest. T comprises tuples that have a time, a label, and a continuation, (t, ℓ, κ). We will use the same stack notation for Q and T as we

$$\begin{array}{rcl} \text{Time } t & \in & \{0, 1, 2, \ldots, \infty\} \\ \text{Label } \ell & ::= & x \mid \mathit{sync} \\ \text{Continuation } \kappa & ::= & \textbf{fn } x_{unit} \rightarrow e \\ \text{Signal Queue } T & ::= & \epsilon \mid (t, \ell, \kappa) \circ T \end{array} \qquad \begin{array}{rcl} \text{Frame } F & ::= & \langle \ell, e \rangle \\ \text{Frame Stack } FS & ::= & \epsilon \mid F \circ FS \\ \text{Process } P & ::= & \{FS_1, \ldots, FS_n\} \\ \text{Task Queue } Q & ::= & \epsilon \mid (\ell, \kappa) \circ Q \end{array}$$

$$e ::= \cdots \mid \text{sys\_block } e \mid \text{sys\_time}() \mid \text{sys\_io}(e, e) \mid \text{sys\_start\_soon}(\overline{\ell, e}) \mid \text{sys\_start\_later}(e, \ell, e)$$

(a) The full grammar for λ_{async}.

C# / JavaScript / Rust / Python

$e ::= \ldots \mid \textbf{async fn } \bar{x} \rightarrow e \mid \textbf{await } e$

$v ::= \ldots \mid \textbf{async fn } \bar{x} \rightarrow e$

Asyncio / Trio / Smol / Tokio

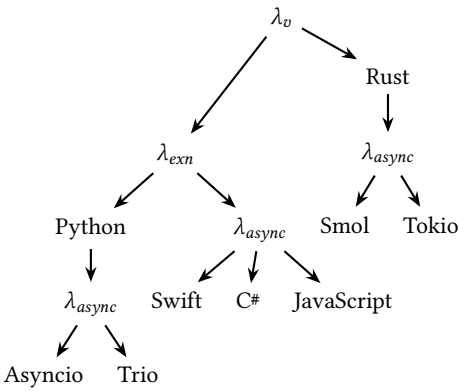
$e ::= \ldots \mid \text{spawn } e \mid \text{cancel } e$

Swift

$e ::= \ldots \mid \textbf{async fn } \bar{x} \rightarrow e \mid \textbf{await } e$
$\mid \text{cancel } e \mid \text{cancelled?}$

$v ::= \ldots \mid \textbf{async fn } \bar{x} \rightarrow e$

(b) The grammar extensions for each lanugage.



(c) The order of extension for each async runtime.

Fig. 13. The grammar extensions and order of extension for λ_{async} and the seven async systems under study.

do for FS. We will additonally write $Q \triangleleft (\ell, \kappa)$ to append to the end of the queue, and $Q_0 +\!\!+ Q_1$ to concatenate two sequences (applicable to both Q and T).

The async platform provides a few additional expressions that we will describe here. The `sys_block` (e) expression provides the barrier between synchronous code and asynchronous code. Languages may use a block expression to block a synchronous frame until the asynchronous expression has completed running. The block expression marks the runtime entry, and when it returns, the runtime exit.

The expression `sys_io`(e_t, e) is the I/O operation provided by the runtime system. It models real-world I/O operations that would take some non-deterministic amount of time to resolve, like receiving packets from the network or querying a database. The form is prescient because the result is provided upfront: e.g., `sys_io`$(5,$ "hello"$)$ creates a task that will resolve to the value "hello" after at least five time steps. Furthermore, we model I/O operations that may not complete as `sys_io`(∞, e). The `sys_io` form is deterministic in its evaluation *value*, but not in its evaluation *time*.

The expression `sys_start_soon`(ℓ, κ) appends the continuation κ to Q, set to run in a frame with the specified label ℓ. The expression `sys_start_later`(t, ℓ, κ) appends the continuation to the signal queue, T, which will be scheduled after timestep t. The expression `sys_time`$()$ simply evaluates to the current timestep.

The grammar extensions for the remaining languages are shown in Figure 12, and the order in which we layer the extensions in Figure 13c. The reduction rules for all async capable languages, which base Python and Rust are not, are defined over the tuple $\langle t, \sigma, Q, T, P \rangle$.

$\langle t_0, \sigma_0, Q, T, P \uplus \{\langle \ell, E[(\textbf{async fn}\ \bar{x} \to e_{body})\ \bar{v}]\rangle \circ FS\}\rangle$ [C♯/JavaScript-Async-App]
$\longrightarrow \langle \text{step}(t_0), \sigma_2, Q, T, P \uplus \{\ \langle x_{task}, e_{task}\rangle\ \circ \langle \ell, E[x_{task}]\rangle \circ FS\}\rangle$
where $(\sigma_1, v_{task}) = \text{task\_alloc}(\sigma_0)$ and $x_{task} = \text{gensym}()$ and $\bar{x}_{fresh} = \overline{\text{gensym}()}$
$\sigma_2 = \sigma_1[x_{task} \mapsto v_{task}, \bar{x}_{fresh} \mapsto \bar{v}]$
$e_{subst} = e_{body}[\bar{x}_{fresh}/\bar{x}]$
$e_{task} = \text{reset}\ ($
try task_set_done(x_{task}, e_{subst}) catch $v_{err} \to$ task_set_failed(x_{task}, v_{err})
sys_start_soon(task_get_dependents(x_{task})))

$\langle t_0, \sigma_0, Q_0, T, P \uplus \{\langle \ell, E[(\textbf{async fn}\ \bar{x} \to e_{body})\ \bar{v}]\rangle \circ FS\}\rangle$ [Swift-Async-App]
$\longrightarrow \langle \text{step}(t_0), \sigma_2,\ Q_0 \lhd (x_{task}, \kappa)\ , T, P \uplus \{\langle \ell, E[x_{task}]\rangle \circ FS\}\rangle$
where $(\sigma_1, x_{task}, v_{task}) =$ task_alloc_dependency(σ_0, ℓ) and $\bar{x}_{fresh} = \overline{\text{gensym}()}$
$\sigma_2 = \sigma_1[x_{task} \mapsto v_{task}, \bar{x}_{fresh} \mapsto \bar{v}]$
$e_{subst} = e_{body}[\bar{x}_{fresh}/\bar{x}]$
$\kappa = \textbf{fn}\ x_{unit} \to x_{unit}\ ;\ \text{reset}\ ($
try task_set_done(x_{task}, e_{subst}) catch $v_{err} \to$ task_set_failed(x_{task}, v_{err});
task_cancel_dependencies(x_{task});
task_wait_on_dependencies(x_{task});
sys_start_soon(task_get_dependents(x_{task})))

$\langle \sigma_0, E[(\textbf{async fn}\ \bar{x} \to e_{body})\ \bar{v}]\rangle \longrightarrow \langle \sigma_1, E[\ \text{reset}\ (\text{shift}\ k \to k\ ;\ e_{subst})\ ]\rangle$ [Rust/Python-Async-App]
where $\bar{x}_{fresh} = \overline{\text{gensym}()}$ and $\sigma_1 = \sigma_0[\bar{x}_{fresh} \mapsto \bar{v}]$
$e_{subst} = e_{body}[\bar{x}_{fresh}/\bar{x}]$

Eagerness Extent Destruction Propagation

Fig. 14. A comparison of the rules [Async-App] across languages.

The goal of the operational semantics is to highlight the differences between languages. We will therefore present the same reduction rule from different languages together. We will also call attention to the portion relevant to each design dimension with a highlight in the following colors: eagerness, extent, destruction, propagation, and suspension, and within cancellation: awareness, direction, and persistence. For space reasons we do not present the rules relevant for task references, and we simplify the presentation with a liberal use of metafunctions; the full definitions of grammars, reductions, and metafunctions are provided in Section A.8.

4.1 Start/End of Life

The rule [Async-App] defines how a coroutine/task is constructed and, for non-lazy languages, what a task does at the end of its computation. We will use the definitions in Figure 14 to highlight the semantics for the start and end of a computation's life.

Figure 14 provides the rule [Async-App] rule for eager, semi-eager, and lazy async applications. For eager and semi-eager languages an async application follows a similar procedure: a new task is allocated, and the function body is placed inside error handling logic to trap exceptions. Where the new expression is placed varies. Eager languages set the expression in a new frame on *top* of the current frame stack. Semi-eager languages place the expression in the task queue to run soon by the next available thread. The semantics for the lazy languages diverge significantly. Neither Rust nor Python has a built-in async runtime, so their semantics are not defined as an extension of

λ_{async}. The lazy [Async-App] variant simply captures the function body as a continuation. We will discuss the [Spawn] rule — used to turn a coroutine into a task — below.

[Async-App] for the (semi-)eager languages also defines the end of life for a task. When C♯/JavaScript allocate a task, it is independent of the current context, meaning that the new task is not a dependency of the executing task (labelled ℓ). When the computation finishes, the task's dependents (those that have awaited it) are immediately scheduled to run. There is an absence of destruction and propagation logic for dependencies, which defines the dynamic extent of tasks for C♯ and JavaScript. Swift, on the other hand, allocates the new task as a *dependency* of the executing task ℓ. After the computation finishes, dependencies are cancelled and waited on. Here we distinguish *waiting on* from *awaiting*; the latter implies that exceptions are propagated, which Swift does not do.

$\langle t_0, \sigma_0, Q_0, T, P \uplus \{\langle \ell, E[\text{spawn } v_{coro}]\rangle \circ FS\}\rangle$ [Asyncio/Tokio/Smol-Spawn]
$\longrightarrow \langle \text{step}(t_0), \sigma_2, Q_0 \lhd (x_{task}, \kappa), T, P \uplus \{\langle \ell, E[x_{task}]\rangle \circ FS\}\rangle$
where $(\sigma_1, x_{task}, v_{task}) = \text{task\_alloc}(\sigma_0)$
$\sigma_2 = \sigma_1[x_{task} \mapsto v_{task}]$
$\kappa = \textbf{fn}\ x_{unit} \to x_{unit}$; reset (
try task_set_done(x_{task}, **await** v_{coro}) catch $v_{err} \to$ task_set_failed(x_{task}, v_{err});
sys_start_soon(task_get_dependents(x_{task})))

$\langle t_0, \sigma_0, Q_0, T, P \uplus \{\langle \ell, E[\text{spawn } v_{coro}]\rangle \circ FS\}\rangle$ [Trio-Spawn]
$\longrightarrow \langle \text{step}(t_0), \sigma_2, Q_0 \lhd (x_{task}, \kappa), T, P \uplus \{\langle \ell, E[x_{task}]\rangle \circ FS\}\rangle$
where $(\sigma_1, x_{task}, v_{task}) =$ task_allocate_dependency(σ_0, ℓ)
$\sigma_2 = \sigma_1[x_{task} \mapsto v_{task}]$
$\kappa = \textbf{fn}\ x_{unit} \to x_{unit}$; reset (
try task_set_done(x_{task}, await v_{coro}) catch $v_{err} \to$ task_set_failed(x_{task}, v_{err});
task_await_dependencies (x_{task})
sys_start_soon(task_get_dependents(x_{task})))

Eagerness Extent Destruction Propagation

Fig. 15. A comparison of [Spawn] across Rust/Python runtimes. The `try/catch` is colored gray because it is present in the Asyncio semantics but not for Tokio/Smol; the rules are otherwise equivalent.

$\langle t_0, \sigma, Q, T, P \uplus \{\langle \ell, E[\textbf{await } v_{aw}]\rangle \circ FS\}\rangle$ [C♯/Swift/Asycio/Trio/Smol-Await]
$\longrightarrow \langle \text{step}(t_0), \sigma, Q, T, P \uplus \{\langle \ell, E[e_{check}]\rangle \circ FS\}\rangle$
where $e_k = \text{shift } k \to \text{task\_add\_waiter}(v_{aw}, \ell, \text{task\_continue\_with}(v_{aw}, k))$
$e_{check} =$ if task_is_completed(v_{aw}) then task_get_result(v_{aw}) else e_k

$\langle t_0, \sigma, Q, T, P \uplus \{\langle \ell, E[\textbf{await } v_{aw}]\rangle \circ FS\}\rangle$ [JavaScript-Await]
$\longrightarrow \langle \text{step}(t_0), \sigma, Q, T, P \uplus \{\langle \ell, E[\ e_k\ ]\rangle \circ FS\}\rangle$
where $e_k = \text{shift } k \to \text{task\_add\_waiter}(v_{aw}, \ell, \text{task\_continue\_with}(v_{aw}, k))$

Suspension

Fig. 16. A comparison of [Await] semantics that define dynamic and static suspension. Dynamic suspending languages only suspend when the awaited value isn't immediately ready.

$$\langle t_0, \sigma, Q, T, P \uplus \{\langle \ell, E[\ \text{cancelled?}()\ ]\rangle \circ FS\}\rangle \qquad \text{[Swift-Cancelled?]}$$
$$\longrightarrow \langle t_0, \sigma, Q, T, P \uplus \{\langle \ell, E[\ \text{task\_cancelled}(\sigma, \ell)\ ]\rangle \circ FS\}\rangle$$

$$\langle t_0, \sigma, (\ell_{waiting}, \kappa) \circ Q_1, T, P \uplus \{\epsilon\}\rangle \qquad \text{[Swift/Trio-Schedule-Cancelled]}$$
$$\longrightarrow \langle \text{step}(t_0),\ \sigma,\ Q_1, T, P \uplus \{\ \langle \ell_{waiting},\ \text{throw\_in}(\kappa, \text{“cancelled”})\ \rangle \circ \epsilon\}\rangle$$
$$\text{where task\_cancelled}(\sigma, \ell_{waiting})$$

$$\langle t_0, \sigma_0, (\ell_{waiting}, \kappa) \circ Q_1, T, P \uplus \{\epsilon\}\rangle \qquad \text{[Asyncio-Schedule-Cancelled]}$$
$$\longrightarrow \langle \text{step}(t_0), \sigma_1, Q_1, T, P \uplus \{\ \langle \ell_{waiting},\ \text{throw\_in}(\kappa, \text{“cancelled”})\ \rangle \circ \epsilon\}\rangle$$
$$\text{where task\_cancelled}(\sigma_0, \ell_{waiting})$$
$$\sigma_1 = \text{task\_uncancel}(\sigma_0, \ell_{waiting})$$

$$\langle t_0, \sigma, (\ell_{waiting}, \kappa) \circ Q_1, T, P \uplus \{\epsilon\}\rangle \longrightarrow \langle \text{step}(t_0),\ \sigma,\ Q_1, T, P \uplus \{\ \epsilon\ \}\rangle \qquad \text{[Tokio-Schedule-Cancelled]}$$
$$\text{where task\_cancelled}(\sigma, \ell_{waiting})$$

$$\langle t_0, \sigma, (\ell_{waiting}, \kappa) \circ Q_1, T, P \uplus \{\epsilon\}\rangle \qquad \text{[Smol-Schedule-Cancelled]}$$
$$\longrightarrow \langle \text{step}(t_0),\ \sigma,\ Q_1, T, P \uplus \{\langle \ell_{waiting}, e_{fail}\rangle \circ \epsilon\}\rangle$$
$$\text{where task\_cancelled}(\sigma, \ell_{waiting})$$
$$e_{fail} = \text{task\_set\_failed}(\ell_{waiting}, ())\ ;\ \text{sys\_start\_soon}(\text{task\_get\_waiters}(x_{task}))$$

Awareness Direction Persistence

Fig. 17. An illustrative subset of cancellation semantics.

The rule [Spawn] is provided by the runtimes of lazy languages to elevate a coroutine to a task registered in the runtime. [Spawn] defines the semantics for the task start and end of life that are absent from the lazy definition of [Async-App].

[Spawn] is defined separately for languages with indefinite and dynamic extent. The definition used by Asyncio/Tokio/Smol allows for indefinite extent and otherwise follows the semantics discussed previously for C♯'s [Async-App] rule.

Trio enforces dynamic extent and the rule follows closely that discussed previously for Swift. However, at the end of execution the task does not cancel its dependencies, but it does *await* them; awaiting dependencies includes propagating an exception.

Figure 16 provides the definitions for the rule [Await]. JavaScript, the only language sampled with static suspension semantics, always captures the current delimited continuation and adds itself as a dependent on the awaited value. Languages with dynamic suspension first check if the awaited value is ready. If it is, no suspension occurs. If a value is not ready, the continuation is captured and registered as a dependent of the pending computation.

4.2 Cancellation

Figure 17 provides the full set of rules for the cancellation mechanism provided by Swift, Tokio, Smol, Asyncio, and Trio. No rules are present for C♯ or JavaScript as they do not provide a built-in mechanism to cancel a running task. The design dimensions for cancellation are awareness, direction, and persistence.

Rules [Cancelled?] and [Schedule-Cancelled] define how a task becomes aware of its cancellation. Swift provides the `cancelled?` function that returns the cancellation status of the current task. The existence of this operation defines cancellation awareness, and simultaneously also the cancellation

direction. A task does not need to rely on another to signal cancellation; this information is communicated between the runtime directly to each task.

The [Schedule-Cancelled] rule for Asyncio/Trio also provides awareness to running tasks that they are being cancelled by injecting an exception into the running computation at the suspended `await` point. Unlike Swift, this cancellation starts from a task's dependencies and is expected to propagate up through dependents. The propagating exception could be caught and handled by an intermediate task, thus suppressing the cancellation.

A peculiarity of Asyncio is the transient persistence of cancellation: after a cancelled task is resumed with a cancellation signal, it is no longer marked as cancelled. Further async work can be done and a second cancellation signal will not be raised. Trio and Swift both have persistent cancellation, so if further async work is done in an exception handler, it would be resumed with another cancellation exception.[5]

The Rust runtimes, Tokio and Smol, do not provide cancellation awareness. Cancelled tasks are not rescheduled but deallocated. The two diverge in API because the `cancel`(e) expression is itself asynchronous. This change is mostly cosmetic and changes neither the cancellation awareness nor direction.

5 Related Work

The study of asynchronous programming sits at the intersection of several bodies of literature. We organize related work into three groups: foundational models of concurrency from which asynchrony draws its semantics, the development of deferred-value abstractions that gave asynchrony its first concrete programming primitives, and the functional and monadic tradition that supplied the compositional framework underlying async/await.

The earliest formal treatments of concurrency were not primarily concerned with asynchrony as we have defined it, but they established the semantic vocabulary on which later work depends. The first formalized models were the actor model [2, 4] and communicating sequential processes [18]. The π-calculus extended this work by allowing first-class channels that can also be sent between processes [29].

Deferred value abstractions have been introduced in many forms and under many aliases. Baker and Hewitt [4] introduced the term *future,* which was also used by Mulitlisp, that extended Scheme with several forms for expressing concurrency [15]. An independent exploration of demand-driven evaluation by Friedman and Wise [13] brought about the lazy *cons* operator. Miller et al. [28] defined *promise pipelining,* rather than blocking to receive a value, a program could register a continuation that would execute upon resolution.

Purer functional languages asked not how to represent a deferred value, but rather how to compose effectful computations while maintaining referential transparency. Jones et al. [20] built concurrent Haskell, which was later extended with shared-state concurrency [16]. The monadic model influenced F♯, the first major async/await design, where the compiler automatically rewrote a block of code written in direct-style into its continuation-passing equivalent [43].

In addition, Bierman et al. [6] published a formal model for Featherweight C♯ (v5) from which we drew much inspiration. The term "structured concurrency" emerged from practitioners in early 2016 [39–41], but the idea of structured and automatic resource management was introduced in Racket's *custodians* [12].

[5] To perform asynchronous cleanup work, runtimes like Trio and Swift will provide cancellation *shields*, which temporarily protect a task against cancellation so that async work can finish.

6 Discussion

The book is not closed on the design of straight-line asynchrony. Communities are actively exploring extensions to the core async/await semantics presented in this work.

Elizarov et al. [11] use the *suspend* keyword in Kotlin to define async functions. However, in lieu of an explicit await keyword, the language automatically introduces suspension points where necessary. They argue that this approach avoids forgotten awaits, and further that it improves the look-and-feel of calling a suspending function, as it looks identical to calling a non-suspending [synchronous] function. However, this claim, and other consequences of this design, are not accompanied by an evaluation.

The Zig programming language is in the process of stabilizing its "colorblind" async/await [7, 21]. The core idea is to parameterize all code over an I/O implementation; libraries can write code to take advantage of asynchrony when available, and library clients can pass any I/O implementation they have available, including a synchronous variant. This approach makes it easier to interoperate sync and async code, but introduces new risks where libraries written explicitly for asynchrony can deadlock and/or panic at runtime if asynchrony is unavailable.

C++ 20 provides a flexible implementation of coroutines that does not ascribe a semantics to its async/await [46]. We cannot attribute C++ to any particular design point in the taxonomy provided in Figure 4 because each axis is configurable. Although elegant and neutral, the choice of full programmability makes each library an async DSL; knowledge transfer between projects within the same language becomes exceedingly difficult.

Where do we go from here? We set out to examine the space of straight-line asynchrony, ourselves confused by some statements we read in individual language descriptions. In particular, as we note in Section 2.1, these languages motivate async from a perspective of similarity to synchrony, which we found difficult to reconcile. Our design space exploration demonstrates the numerous ways in which this analogy fails.

The principal use for our exploration is to help developers. Developers transitioning from one language to another carry their prior knowledge to make sense of the new in terms of the old. It is then especially confusing when the new language has the same keywords and similar description as the old one, but with a different semantics. We believe that our articulation of the design space, such as Figure 4, can help both learners and their educators talk more effectively about how to translate knowledge between languages. We also believe that our exploration can help languages designers in building new features for straight-line asynchrony. Our design space can help enumerate key design dimensions, preventing important details from slipping through the cracks of the design process. Our design space can also help designers more effectively compare and contrast their design against related languages.

An open challenge for the future is to build a body of empirical data that can support designers in reasoning about the trade-offs of different approaches. For example, how much overhead does semi-eager async function application incur versus eager, and how much more concurrency does it gain? To what extent does simultaneous cancellation prevent realistic bugs that would occur with a bottom-up cancellation strategy? Which of these semantics best matches developer expectations for the behavior of asynchronous code? Which decisions lead to more errors? We hope that this design space exploration can inspire practitioners and academics alike to build a better understanding of the evolving world of asynchronous programming.

Data-Availability Statement

The paper's artifact will contain two main components:

(1) All the code associated with the examples in the paper, runnable in their respective languages.
(2) The full suite of Redex models providing the described semantics.

References

[1] ECMA TC 39. 2016. TC39 — Proposal async-await. https://github.com/tc39/proposal-async-await original-date: 2014-01-28T22:24:00Z.

[2] Gul Agha. 1986. *Actors: A Model of Concurrent Computation in Distributed Systems.* The MIT Press. doi:10.7551/mitpress/1086.001.0001

[3] Jake Archibald. 2023. The gotcha of unhandled promise rejections. https://jakearchibald.com/2023/unhandled-rejections/

[4] Henry C. Baker and Carl Hewitt. 1977. The incremental garbage collection of processes. In *Proceedings of the 1977 symposium on Artificial intelligence and programming languages.* Association for Computing Machinery, New York, NY, USA, 55–59. doi:10.1145/800228.806932

[5] Jacob Bartlett. 2023. Why is my Task running on the main thread? https://blog.jacobstechtavern.com/p/why-is-task-running-on-main-thread

[6] Gavin Bierman, Claudio Russo, Geoffrey Mainland, Erik Meijer, and Mads Torgersen. 2012. Pause 'n' Play: Formalizing Asynchronous C#. In *ECOOP 2012 – Object-Oriented Programming.* Vol. 7313. Springer Berlin Heidelberg, Berlin, Heidelberg, 233–257. doi:10.1007/978-3-642-31057-7_12

[7] Loris Cro. 2020. What is Zig's "Colorblind" Async/Await? https://kristoff.it/blog/zig-colorblind-async-await/

[8] M. a m a D. 2025. Why only creating a task will run the coroutine in python? https://stackoverflow.com/q/79755548

[9] Olivier Danvy and Andrzej Filinski. [n. d.]. A Functional Abstraction of Typed Contexts. ([n. d.]).

[10] ECMA Ecma. [n. d.]. ECMAScript® 2026 Language Specification. https://262.ecma-international.org/16.0

[11] Roman Elizarov, Mikhail Belyaev, Marat Akhin, and Ilmir Usmanov. 2021. Kotlin coroutines: design and implementation. In *Proceedings of the 2021 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software.* ACM, Chicago IL USA, 68–84. doi:10.1145/3486607.3486751

[12] Matthew Flatt, Robert Bruce Findler, Shriram Krishnamurthi, and Matthias Felleisen. 1999. Programming languages as operating systems (or revenge of the son of the Lisp machine). *SIGPLAN Not.* 34, 9 (Sept. 1999), 138–147. doi:10.1145/317765.317793

[13] Daniel P. Friedman and David S. Wise. 1976. CONS Should Not Evaluate Its Arguments. In *Proceedings of the Third International Colloquium on Automata, Languages and Programming (ICALP 1976).* Edinburgh University Press, Edinburgh, Scotland, 257–284.

[14] Al Gebra. 2023. How can I start a Python async coroutine eagerly? https://stackoverflow.com/q/69145007

[15] Robert H. Halstead. 1985. MULTILISP: a language for concurrent symbolic computation. *ACM Trans. Program. Lang. Syst.* 7, 4 (Oct. 1985), 501–538. doi:10.1145/4472.4478

[16] Tim Harris, Simon Marlow, Simon Peyton Jones, and Maurice Herlihy. [n. d.]. Composable Memory Transactions. ([n. d.]).

[17] Alexander Hartl. 2022. asyncio: Use strong references for free-flying tasks · Issue #91887 · python/cpython. https://github.com/python/cpython/issues/91887

[18] C A R Hoare. 1978. Communicating sequential processes. 21, 8 (1978).

[19] Apple Inc. [n. d.]. Embracing Swift concurrency - WWDC25 - Videos. https://developer.apple.com/videos/play/wwdc2025/268/

[20] Simon Peyton Jones, Andy Gordon, and Sigbjorn Finne. 1996. Concurrent Haskell. 295–308. https://www.microsoft.com/en-us/research/publication/concurrent-haskell/

[21] Andrew Kelley. 2025. Zig's New Async I/O (Text Version) - Andrew Kelley. https://andrewkelley.me/post/zig-new-async-io-text-version.html

[22] Loris Lacuna. [n. d.]. asyncio: a library with too many sharp corners. https://sailor.li/asyncio.html

[23] Kuang-Chen Lu and Shriram Krishnamurthi. 2024. Identifying and Correcting Programming Language Behavior Misconceptions. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (April 2024), 334–361. doi:10.1145/3649823

[24] MaPePeR. 2023. Starting or resuming async method/coroutine without awaiting it in Python. https://stackoverflow.com/q/77667669

[25] John McCall and Doug Gregor. 2020. Swift Evolution Proposals — 0296: async/await. https://github.com/swiftlang/swift-evolution/blob/main/proposals/0296-async-await.md

[26] John McCall, Doug Gregor, and Konrad Malawski. 2021. Swift Evoluation Proposals — 0317: async let. https://github.com/swiftlang/swift-evolution/blob/main/proposals/0317-async-let.md

[27] John McCall, Joe Groff, Doug Gregor, and Konrad Malawski. 2021. Swift Evolution Proposals —0304: Structured Concurrency. https://github.com/swiftlang/swift-evolution/blob/main/proposals/0304-structured-concurrency.md

[28] Mark S. Miller, E. Dean Tribble, and Jonathan Shapiro. 2005. Concurrency Among Strangers. In *Trustworthy Global Computing.* Vol. 3705. Springer Berlin Heidelberg, Berlin, Heidelberg, 195–229. doi:10.1007/11580850_12 Series Title: Lecture Notes in Computer Science.

[29] Robin Milner, Joachim Parrow, and David Walker. 1992. A calculus of mobile processes, I. *Information and Computation* 100, 1 (Sept. 1992), 1–40. doi:10.1016/0890-5401(92)90008-4
[30] Ana Lúcia De Moura and Roberto Ierusalimschy. 2009. Revisiting coroutines. *ACM Transactions on Programming Languages and Systems* 31, 2 (Feb. 2009), 1–31. doi:10.1145/1462166.1462167
[31] David Pacheco. [n. d.]. 397 - Challenges with async/await in the control plane / RFD / Oxide. https://rfd.shared.oxide.computer/rfd/397
[32] Rain Paharia. 2025. 400 - Dealing with cancel safety in async Rust / RFD / Oxide. https://rfd.shared.oxide.computer/rfd/400
[33] Rain Paharia. 2025. Cancelling async Rust · sunshowers. https://sunshowers.io/posts/cancelling-async-rust/
[34] Saad. 2021. Using async/await with a forEach loop. https://stackoverflow.com/q/37576685
[35] Yury Selivanov. 2015. PEP 492 – Coroutines with async and await syntax | peps.python.org. https://peps.python.org/pep-0492/
[36] Nathanial J Smith. 2016. Some thoughts on asynchronous API design in a post-async/await world — njs blog. https://vorpus.org/blog/some-thoughts-on-asynchronous-api-design-in-a-post-asyncawait-world/
[37] Nathanial J Smith. 2018. Notes on structured concurrency, or: Go statement considered harmful. https://vorpus.org/blog/notes-on-structured-concurrency-or-go-statement-considered-harmful/
[38] ECMA C# standard committee. [n. d.]. C# Language Specification. https://learn.microsoft.com/en-us/dotnet/csharp/specification
[39] Martin Sustrik. 2016. Getting rid of state machines (I). https://sustrik.github.io/250bpm/blog:69/
[40] Martin Sustrik. 2016. Getting rid of state machines (II). https://sustrik.github.io/250bpm/blog:70/
[41] Martin Sustrik. 2025. Structured Concurrency. https://www.250bpm.com/p/structured-concurrency
[42] Don Syme, Tomas Petricek, and Dmitry Lomov. 2011. The F# Asynchronous Programming Model. In *Practical Aspects of Declarative Languages*, Ricardo Rocha and John Launchbury (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 175–189.
[43] Don Syme, Tomas Petricek, and Dmitry Lomov. 2011. The F# asynchronous programming model. In *Proceedings of the 13th international conference on Practical aspects of declarative languages (PADL'11)*. Springer-Verlag, Berlin, Heidelberg, 175–189.
[44] Mads Torgersen. [n. d.]. Asynchronous Programming in C# and Visual Basic. ([n. d.]).
[45] Bill Wagner. 2025. Asynchronous programming - C#. https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/
[46] ISO/IEC JTC1 SC22 WG21. 2017. Technical Specification for {C++} Extensions for Coroutines. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4680.pdf
[47] William. 2019. How does javascript async/await actually work? https://stackoverflow.com/q/57160341
[48] Grant Winney. 2021. Using Async, Await, and Task to keep the WinForms UI responsive. https://grantwinney.com/using-async-await-and-task-to-keep-the-winforms-ui-more-responsive/ Section: posts.
[49] withoutboats. 2018. 2394 — async/await - The Rust RFC Book. https://rust-lang.github.io/rfcs/2394-async_await.html
[50] withoutboats. 2023. Why async Rust? https://without.boats/blog/why-async-rust/